

A Summary of Offline Reinforcement Learning Methods

Name: Hongming Zhang

Date: 2022.10.14

Introduction

Offline reinforcement learning (RL) algorithms utilize previously collected data, without additional online data collection.

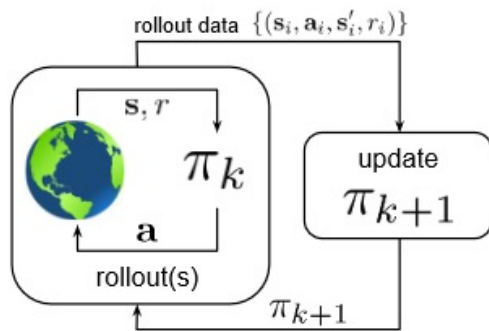
This slides **summarize some popular offline RL algorithms** in recent years.

- Introduction to offline RL
- Model-free offline RL
- Model-based offline RL

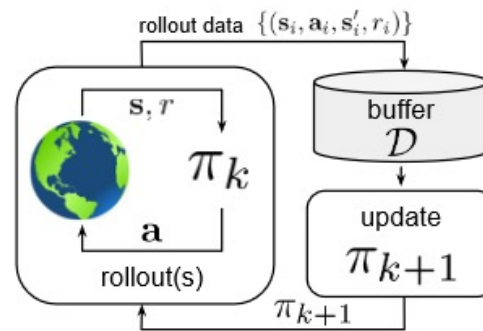
Offline Reinforcement Learning

The learning algorithm is provided with a **static dataset** of transitions, $\mathcal{D} = \{(s_t^i, a_t^i, s_{t+1}^i, r_t^i)\}$ and must **learn the best policy** it can using this dataset.

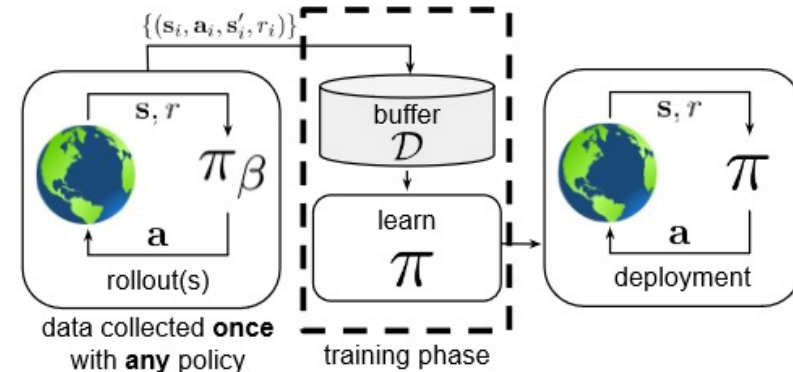
(a) online reinforcement learning



(b) off-policy reinforcement learning



(c) offline reinforcement learning



What Makes Offline Reinforcement Learning Difficult?

- Exploration is outside the scope of the algorithm
- **Distributional shift**: while our function approximator (policy, value function, or model) might be trained under one distribution, it will be evaluated on a different distribution, due both to the change in visited **states** for the new policy and, more subtly, by the **act** of maximizing the expected return (**extrapolation error**).

Model-Free Offline RL

- BCQ (Batch-Constrained Deep Q-Learning)
- BEAR (Bootstrapping Error Accumulation Reduction)
- BRAC (Behavior Regularized Actor Critic)
- REM (Random Ensemble Mixture)
- CQL (Conservative Q-Learning)
- AWAC (Advantage Weighted Actor Critic)
- TD3+BC
- IQL (Implicit Q-Learning)

Restricts the action space in order to force the agent towards **behaving close to on-policy** with respect to a subset of the given data.

- **Generative model** G_ω : generates candidate actions with high similarity to the batch.
- **Perturbation model** $\xi_\phi(s, a, \Phi)$: outputs an adjustment to an action a in the range $[-\Phi, \Phi]$.
- Two **Q networks**, $Q_{\theta_1}(s, a)$, $Q_{\theta_2}(s, a)$: Clipped Double Q-learning/TD3

$$\pi(s) = \underset{a_i + \xi_\phi(s, a_i, \Phi)}{\operatorname{argmax}} Q_\theta(s, a_i + \xi_\phi(s, a_i, \Phi)),$$

$$\{a_i \sim G_\omega(s)\}_{i=1}^n.$$

$$r + \gamma \max_{a_i} \left[\lambda \min_{j=1,2} Q_{\theta'_j}(s', a_i) + (1 - \lambda) \max_{j=1,2} Q_{\theta'_j}(s', a_i) \right]$$

The choice of n and Φ creates a trade-off between an imitation learning and reinforcement learning algorithm.

Algorithm 1 BCQ

Input: Batch $\mathcal{B}$, horizon T , target network update rate τ , mini-batch size N , max perturbation Φ , number of sampled actions n , minimum weighting λ .

Initialize Q-networks $Q_{\theta_1}, Q_{\theta_2}$, perturbation network ξ_ϕ , and VAE $G_\omega = \{E_{\omega_1}, D_{\omega_2}\}$, with random parameters $\theta_1, \theta_2, \phi, \omega$, and target networks $Q_{\theta'_1}, Q_{\theta'_2}, \xi_{\phi'}$ with $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$.

for $t = 1$ **to** T **do**

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\mu, \sigma = E_{\omega_1}(s, a), \quad \tilde{a} = D_{\omega_2}(s, z), \quad z \sim \mathcal{N}(\mu, \sigma)$

$\omega \leftarrow \operatorname{argmin}_\omega \sum (a - \tilde{a})^2 + D_{\text{KL}}(\mathcal{N}(\mu, \sigma) || \mathcal{N}(0, 1))$

 Sample n actions: $\{a_i \sim G_\omega(s')\}_{i=1}^n$

 Perturb each action: $\{a_i = a_i + \xi_\phi(s', a_i, \Phi)\}_{i=1}^n$

 Set value target y (Eqn. 13)

$\theta \leftarrow \operatorname{argmin}_\theta \sum (y - Q_\theta(s, a))^2$

$\phi \leftarrow \operatorname{argmax}_\phi \sum Q_{\theta_1}(s, a + \xi_\phi(s, a, \Phi)), a \sim G_\omega(s)$

 Update target networks: $\theta'_i \leftarrow \tau\theta + (1 - \tau)\theta'_i$

$\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$

end for

Support-set matching: restrict the policy to ensure that it output **actions that lie in the support** of the training distribution to reduce bootstrapping error.

- Use K Q -functions and use the minimum Q -value for policy improvement

We denote the set of Q -functions as: $\hat{Q}_1, \dots, \hat{Q}_K$. Then, the policy is updated to maximize the conservative estimate of the Q -values within Π_ϵ :

$$\pi_\phi(s) := \max_{\pi \in \Pi_\epsilon} \mathbb{E}_{a \sim \pi(\cdot|s)} \left[\min_{j=1, \dots, K} \hat{Q}_j(s, a) \right]$$

- Restrict the policy

Putting everything together, the optimization problem in the policy improvement step is

$$\pi_\phi := \max_{\pi \in \Delta_{|S|}} \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{a \sim \pi(\cdot|s)} \left[\min_{j=1, \dots, K} \hat{Q}_j(s, a) \right] \quad \text{s.t.} \quad \mathbb{E}_{s \sim \mathcal{D}} [\text{MMD}(\mathcal{D}(s), \pi(\cdot|s))] \leq \varepsilon \quad (1)$$

where ε is an approximately chosen threshold. We choose a threshold of $\varepsilon = 0.05$ in our experiments.

Algorithm 1 BEAR Q-Learning (BEAR-QL)

input : Dataset $\mathcal{D}$, target network update rate τ , mini-batch size N , sampled actions for MMD n , minimum λ

1: Initialize Q -ensemble $\{Q_{\theta_i}\}_{i=1}^K$, actor π_ϕ , Lagrange multiplier α , target networks $\{Q_{\theta'_i}\}_{i=1}^K$, and a target actor $\pi_{\phi'}$, with $\phi' \leftarrow \phi$, $\theta'_i \leftarrow \theta_i$

2: **for** t in $\{1, \dots, N\}$ **do**

3: Sample mini-batch of transitions $(s, a, r, s') \sim \mathcal{D}$

Q-update:

4: Sample p action samples, $\{a_i \sim \pi_{\phi'}(\cdot|s')\}_{i=1}^p$

5: Define $y(s, a) := \max_{a_i} [\lambda \min_{j=1, \dots, K} Q_{\theta'_j}(s', a_i) + (1 - \lambda) \max_{j=1, \dots, K} Q_{\theta'_j}(s', a_i)]$

6: $\forall i, \theta_i \leftarrow \arg \min_{\theta_i} (Q_{\theta_i}(s, a) - (r + \gamma y(s, a)))^2$

Policy-update:

7: Sample actions $\{\hat{a}_i \sim \pi_\phi(\cdot|s)\}_{i=1}^m$ and $\{a_j \sim \mathcal{D}(s)\}_{j=1}^n$, n preferably an intermediate integer (1-10)

8: Update ϕ , α by minimizing Equation (1) by using dual gradient descent with Lagrange multiplier α

9: **Update Target Networks:** $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$; $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

10: **end for**

A general algorithmic framework which covers existing approaches

- Penalty in the value function

$$\min_{Q_\psi} \mathbb{E}_{\substack{(s,a,r,s') \sim \mathcal{D} \\ a' \sim \pi_\theta(\cdot|s')}} \left[\left(r + \gamma \left(\bar{Q}(s', a') - \alpha \hat{D}(\pi_\theta(\cdot|s'), \pi_b(\cdot|s')) \right) - Q_\psi(s, a) \right)^2 \right]$$

- Penalty solely on the policy

$$\hat{D}(\pi_\theta(\cdot|s'), \pi_b(\cdot|s'))$$

Kernel MMD, KL Divergence, Wasserstein Distance, etc.

BEAR, BCQ, KL-control

A robust Q-learning algorithm that enforces optimal Bellman consistency on **random convex combinations of multiple Q-value estimates**.

- An optimistic perspective on offline RL: with sufficiently **large and diverse datasets**, robust RL algorithms, **without an explicit correction** for distribution mismatch, can result in **high quality policies**.

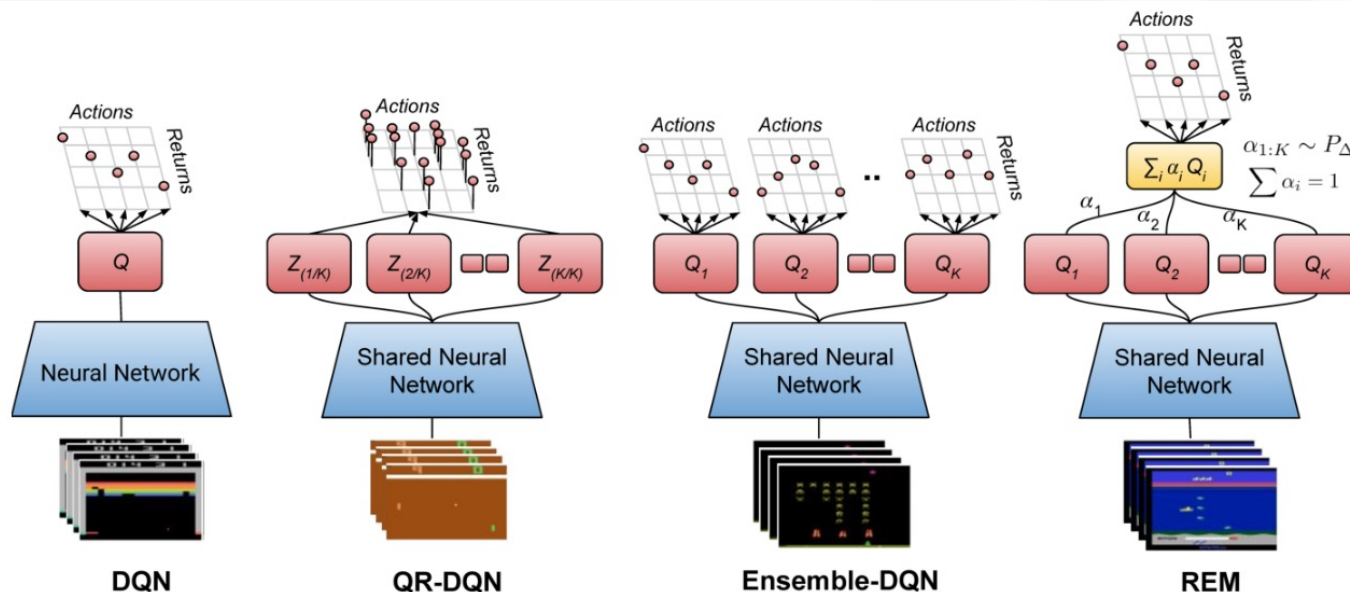


Figure 2: Neural network architectures for DQN, distributional QR-DQN and the proposed expected RL variants, *i.e.*, Ensemble-DQN and REM, with the same multi-head architecture as QR-DQN. The individual Q -heads share all of the neural network layers except the final fully connected layer. In QR-DQN, each head (red rectangles) corresponds to a specific quantile of the return distribution, while in the proposed variants, each head approximates the optimal Q -function.

Augments the standard Bellman error objective with a simple **Q-value regularizer**, to **avoid overestimation** of values induced by the distributional shift between the dataset and the learned policy.

- Bellman operator

$$\mathcal{B}^\pi Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s'|s, a), a' \sim \pi(a'|s')} [Q(s', a')]$$

- Bellman optimality operator

$$\mathcal{B}^* Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s'|s, a)} [\max_{a'} Q(s', a')]$$

- Objective

$$\begin{aligned} \min_Q \max_{\mu} \alpha \left(\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \right) \\ + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right] + \mathcal{R}(\mu) \quad (\text{CQL}(\mathcal{R})). \quad (3) \end{aligned}$$

Algorithm 1 Conservative Q-Learning (both variants)

- 1: Initialize Q-function, Q_θ , and optionally a policy, π_ϕ .
 - 2: **for** step t in $\{1, \dots, N\}$ **do**
 - 3: Train the Q-function using G_Q gradient steps on objective from Equation 4
 $\theta_t := \theta_{t-1} - \eta_Q \nabla_{\theta} \text{CQL}(\mathcal{R})(\theta)$
 (Use $\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic)
 - 4: (only with actor-critic) Improve policy π_ϕ via G_π gradient steps on ϕ with SAC-style entropy regularization:
 $\phi_t := \phi_{t-1} + \eta_\pi \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_\phi(\cdot|\mathbf{s})} [Q_\theta(\mathbf{s}, \mathbf{a}) - \log \pi_\phi(\mathbf{a}|\mathbf{s})]$
 - 5: **end for**
-

Constrains the policy to **prevent offline instability**, but **not so conservative**.

- Employs an implicit constraint, which does not require any explicit modeling of the behavior policy.

- Objective

$$\begin{aligned} \pi_{k+1} = \arg \max_{\pi \in \Pi} \mathbb{E}_{\mathbf{a} \sim \pi(\cdot|\mathbf{s})} [A^{\pi_k}(\mathbf{s}, \mathbf{a})] \\ \text{s.t. } D_{\text{KL}}(\pi(\cdot|\mathbf{s}) || \pi_{\beta}(\cdot|\mathbf{s})) \leq \epsilon. \end{aligned}$$

- Objective with implicit constraint

$$\begin{aligned} \mathcal{L}(\pi, \lambda) &= \mathbb{E}_{\mathbf{a} \sim \pi(\cdot|\mathbf{s})} [A^{\pi_k}(\mathbf{s}, \mathbf{a})] + \lambda(\epsilon - D_{\text{KL}}(\pi(\cdot|\mathbf{s}) || \pi_{\beta}(\cdot|\mathbf{s}))) \\ \theta_{k+1} &= \arg \max_{\theta} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \beta} \left[\log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \exp \left(\frac{1}{\lambda} A^{\pi_k}(\mathbf{s}, \mathbf{a}) \right) \right] \end{aligned}$$

Algorithm 1 Advantage Weighted Actor Critic (AWAC)

```

1: Dataset  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)_j\}$ 
2: Initialize buffer  $\beta = \mathcal{D}$ 
3: Initialize  $\pi_{\theta}, Q_{\phi}$ 
4: for iteration  $i = 1, 2, \dots$  do
5:   Sample batch  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r) \sim \beta$ 
6:    $y = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}', \mathbf{a}'} [Q_{\phi_{k-1}}(\mathbf{s}', \mathbf{a}')]
7:    $\phi \leftarrow \arg \min_{\phi} \mathbb{E}_{\mathcal{D}} [(Q_{\phi}(\mathbf{s}, \mathbf{a}) - y)^2]$ 
8:    $\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \beta} [\log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \exp (\frac{1}{\lambda} A^{\pi_k}(\mathbf{s}, \mathbf{a}))]$ 
9:   if  $i > \text{num\_offline\_steps}$  then
10:     $\tau_1, \dots, \tau_K \sim p_{\pi_{\theta}}(\tau)$ 
11:     $\beta \leftarrow \beta \cup \{\tau_1, \dots, \tau_K\}$ 
12:   end if
13: end for$ 
```

Can we make a deep RL algorithm work offline with minimal changes?

- Simply add a behavior cloning term to regularize the policy

$$\pi = \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\lambda Q(s, \pi(s)) - (\pi(s) - a)^2 \right]$$

- improves the stability of the learned policy by normalizing the states over the dataset

$$s_i = \frac{s_i - \mu_i}{\sigma_i + \epsilon},$$

- Parameter

$$\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|}$$

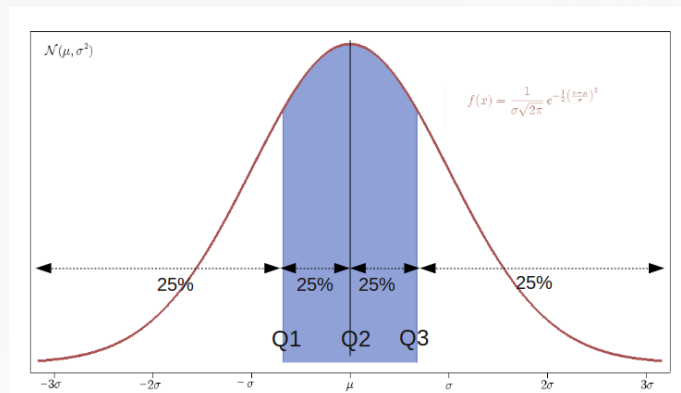
Entirely avoid querying out-of-sample (unseen) actions in our TD loss

- Q-learning

$$L_{TD}(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [(r(s,a) + \gamma \max_{a'} Q_{\hat{\theta}}(s', a') - Q_{\theta}(s,a))^2],$$

- SARSA

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [(r(s,a) + \gamma Q_{\hat{\theta}}(s', a') - Q_{\theta}(s,a))^2].$$



- Estimate the maximum Q-value over actions that are in the support of the data distribution

$$L(\theta) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [(r(s,a) + \gamma \max_{\substack{a' \in \mathcal{A} \\ \text{s.t. } \pi_{\beta}(a'|s') > 0}} Q_{\hat{\theta}}(s', a') - Q_{\theta}(s,a))^2]. \quad (4)$$

- Expectiles/Quantiles regression

Expectile regression provides us with a powerful framework to estimate statistics of a random variable **beyond mean regression**. We can use expectile regression to modify the policy evaluation objective in Eqn. (2) to predict an upper expectile of the TD targets that approximates the maximum of $r(s,a) + \gamma Q_{\hat{\theta}}(s', a')$ **over actions a' constrained to the dataset actions**, as in Eqn. (4). This leads to the following expectile regression objective:

$$L(\theta) = \mathbb{E}_{(s,a,s',a') \sim \mathcal{D}} [L_2^{\tau}(r(s,a) + \gamma Q_{\hat{\theta}}(s', a') - Q_{\theta}(s,a))].$$

Model-Based Offline RL

- Decision Transformer
- Trajectory Transformer
- MOPO (Model-Based Offline Policy Optimization)
- MOREL (Model-Based Offline Reinforcement Learning)
- COMBO (Conservative Offline Model-Based Policy Optimization)
- DAC-MDP (Deep Averagers with Costs MDP)
- MuZero Unplugged
- EfficientZero

Decision Transformer

Casts the problem of RL as **conditional sequence modeling**, simply outputs the optimal actions by leveraging a causally masked Transformer.

- **Bypass** the need for **bootstrapping** for long term credit assignment.
- Achieve policy improvement **without the need for dynamic programming**, matches or exceeds the performance of state-of-the-art model-free offline RL algorithms.

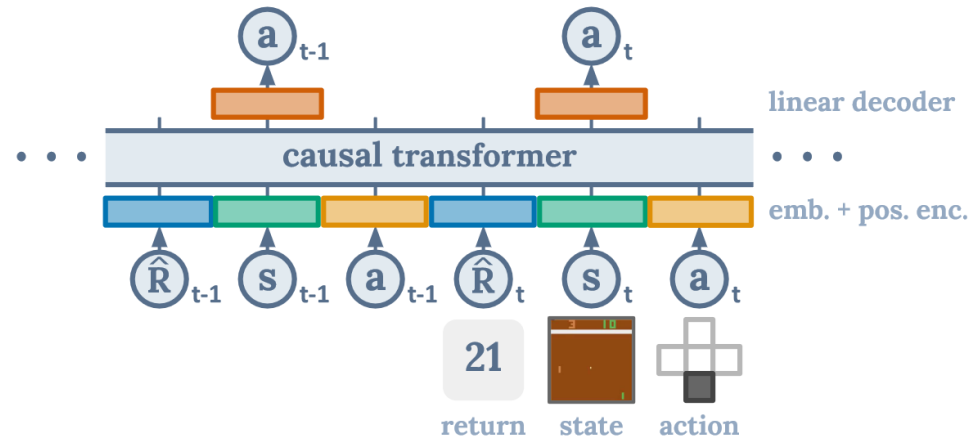


Figure 1: Decision Transformer architecture^[1]. States, actions, and returns are fed into modality-specific **linear embeddings** and a **positional episodic timestep encoding** is added. **Tokens** are fed into a GPT architecture which predicts actions autoregressively using a causal self-attention mask.

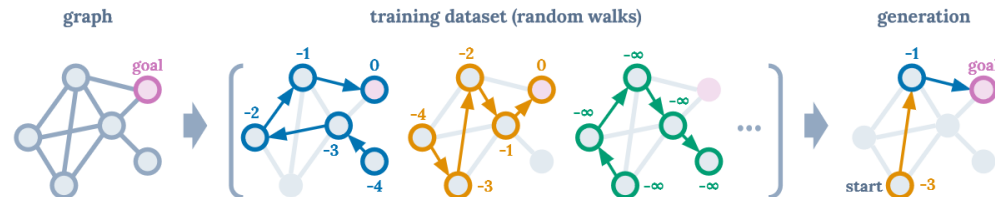


Figure 2: Illustrative example of finding shortest path for a fixed graph (left) posed as reinforcement learning. Training dataset consists of random walk trajectories and their per-node returns-to-go (middle). Conditioned on a starting state and generating largest possible return at each node, Decision Transformer sequences optimal paths.

Trajectory Transformer

Transformer + beam search

- Aim to replace as much of the RL pipeline as possible with sequence modeling, so as to produce a simpler method whose effectiveness is determined by the **representational capacity of the sequence model** rather than algorithmic sophistication.
- **No longer require separate behavior policy constraints**, as is common in prior work on offline model-free RL, and we no longer require ensembles or other epistemic uncertainty estimators.

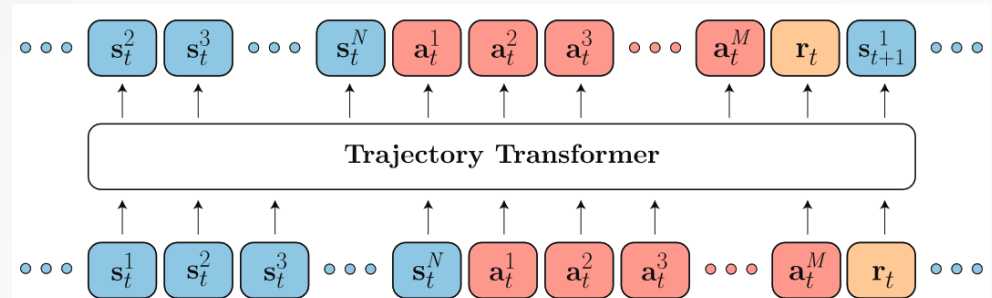


Figure 1 (Architecture) The Trajectory Transformer trains on sequences of (autoregressively discretized) states, actions, and rewards. Planning with the Trajectory Transformer mirrors the sampling procedure used to generate sequences from a language model.

Algorithm 1 Beam search

```

1: Require Input sequence  $\mathbf{x}$ , vocabulary  $\mathcal{V}$ , sequence length  $T$ , beam width  $B$ 
2: Initialize  $Y_0 = \{ () \}$ 
3: for  $t = 1, \dots, T$  do
4:    $\mathcal{C}_t \leftarrow \{ \mathbf{y}_{t-1} \circ y \mid \mathbf{y}_{t-1} \in Y_{t-1} \text{ and } y \in \mathcal{V} \}$  // candidate single-token extensions
5:    $Y_t \leftarrow \underset{Y \subseteq \mathcal{C}_t, |Y|=B}{\operatorname{argmax}} \log P_\theta(Y \mid \mathbf{x})$  //  $B$  most likely sequences from candidates
6: end for
7: Return  $\underset{y \in Y_T}{\operatorname{argmax}} \log P_\theta(y \mid \mathbf{x})$ 

```

Disentangle the task horizon and model horizon by **querying the model only for short rollouts**.

- Combination of model ensembles with short model rollouts is sufficient to prevent model exploitation.

Algorithm 2 Model-Based Policy Optimization with Deep Reinforcement Learning

```

1: Initialize policy  $\pi_\phi$ , predictive model  $p_\theta$ , environment dataset  $\mathcal{D}_{\text{env}}$ , model dataset  $\mathcal{D}_{\text{model}}$ 
2: for  $N$  epochs do
3:   Train model  $p_\theta$  on  $\mathcal{D}_{\text{env}}$  via maximum likelihood
4:   for  $E$  steps do
5:     Take action in environment according to  $\pi_\phi$ ; add to  $\mathcal{D}_{\text{env}}$ 
6:     for  $M$  model rollouts do
7:       Sample  $s_t$  uniformly from  $\mathcal{D}_{\text{env}}$ 
8:       Perform  $k$ -step model rollout starting from  $s_t$  using policy  $\pi_\phi$ ; add to  $\mathcal{D}_{\text{model}}$ 
9:     for  $G$  gradient updates do
10:      Update policy parameters on model data:  $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi, \mathcal{D}_{\text{model}})$ 

```

Can we develop an offline RL algorithm that **generalizes beyond the state and action support** of the offline data?

- Modify MBPO by applying them with **rewards penalized by the uncertainty** of the dynamics. Maximize a lower bound of the return in the true MDP.

Algorithm 1 Framework for Model-based Offline Policy Optimization (MOPO) with Reward Penalty

Require: Dynamics model $\hat{T}$ with admissible error estimator $u(s, a)$; constant λ .

- 1: Define $\tilde{r}(s, a) = r(s, a) - \lambda u(s, a)$. Let $\tilde{M}$ be the MDP with dynamics $\hat{T}$ and reward $\tilde{r}$.
 - 2: Run any RL algorithm on $\tilde{M}$ until convergence to obtain $\hat{\pi} = \arg\max_{\pi} \eta_{\tilde{M}}(\pi)$
-

Algorithm 2 MOPO instantiation with regularized probabilistic dynamics and ensemble uncertainty

Require: reward penalty coefficient λ rollout horizon h , rollout batch size b .

- 1: Train on batch data $\mathcal{D}_{\text{env}}$ an ensemble of N probabilistic dynamics $\{\hat{T}^i(s', r | s, a) = \mathcal{N}(\mu^i(s, a), \Sigma^i(s, a))\}_{i=1}^N$.
 - 2: Initialize policy π and empty replay buffer $\mathcal{D}_{\text{model}} \leftarrow \emptyset$.
 - 3: **for** epoch $1, 2, \dots$ **do** ▷ This for-loop is essentially one outer iteration of MBPO
 - 4: **for** $1, 2, \dots, b$ (in parallel) **do**
 - 5: Sample state s_1 from $\mathcal{D}_{\text{env}}$ for the initialization of the rollout.
 - 6: **for** $j = 1, 2, \dots, h$ **do**
 - 7: Sample an action $a_j \sim \pi(s_j)$.
 - 8: Randomly pick dynamics $\hat{T}$ from $\{\hat{T}^i\}_{i=1}^N$ and sample $s_{j+1}, r_j \sim \hat{T}(s_j, a_j)$.
 - 9: Compute $\tilde{r}_j = r_j - \lambda \max_{i=1}^N \|\Sigma^i(s_j, a_j)\|_{\text{F}}$.
 - 10: Add sample $(s_j, a_j, \tilde{r}_j, s_{j+1})$ to $\mathcal{D}_{\text{model}}$.
 - 11: Drawing samples from $\mathcal{D}_{\text{env}} \cup \mathcal{D}_{\text{model}}$, use SAC to update π .
-

Learns a **pessimistic MDP** (P-MDP) using the offline dataset, then learns a near-optimal policy in this P-MDP

- Using the predictions of ensembles of models to partition the state space into “known” and “unknown” regions

$$\text{disc}(s, a) = \max_{i,j} \|f_{\phi_i}(s, a) - f_{\phi_j}(s, a)\|_2$$

$$U_{\text{practical}}(s, a) = \begin{cases} \text{FALSE (i.e. Known)} & \text{if } \text{disc}(s, a) \leq \text{threshold} \\ \text{TRUE (i.e. Unknown)} & \text{if } \text{disc}(s, a) > \text{threshold} \end{cases}$$

- uses a **large negative reward** for unknown regions.

$$r = \begin{cases} -\kappa, & \text{if } s \text{ is unknown} \\ r(s, a), & \text{otherwise} \end{cases}$$

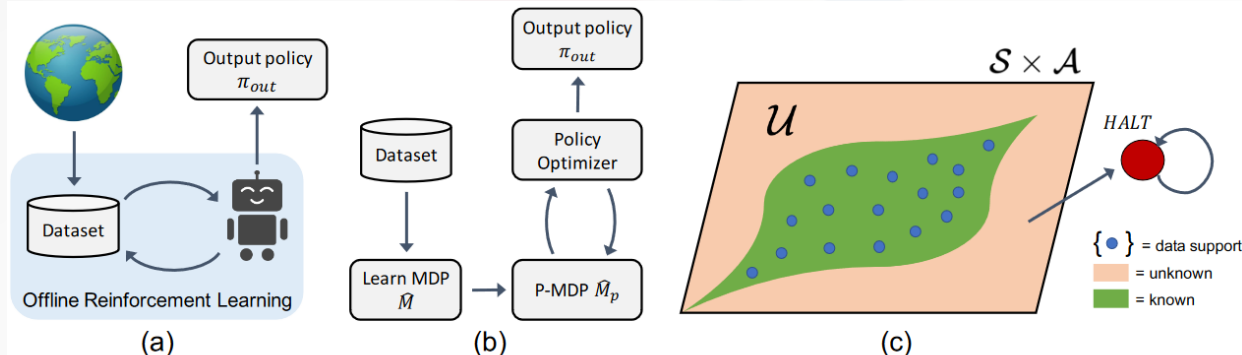


Figure 1: (a) Illustration of the offline RL paradigm. (b) Illustration of our framework, MOReL, which learns a pessimistic MDP (P-MDP) from the dataset and uses it for policy search. (c) Illustration of the P-MDP, which partitions the state-action space into known (green) and unknown (orange) regions, and also forces a transition to a low reward absorbing state (HALT) from unknown regions. Blue dots denote the support in the dataset. Function approximation and generalization allows us to learn about states not numerically identical to data support. See algorithm 1 for more details.

Does **not** assume **access to a model error or uncertainty**.

- learns a conservative critic function using both the offline dataset and data generated using rollouts under the model, by **penalizing** the value function in **state-action tuples that are not in the support** of the offline dataset

Conservative Policy Evaluation: Given a policy π , an offline dataset $\mathcal{D}$, and a learned model of the MDP $\hat{\mathcal{M}}$, the goal in this step is to obtain a conservative estimate of Q^π . To achieve this, we penalize the Q-values evaluated on data drawn from a particular state-action distribution that is more likely to be **out-of-support** while pushing up the Q-values on state-action pairs that are trustworthy, which is implemented by repeating the following recursion:

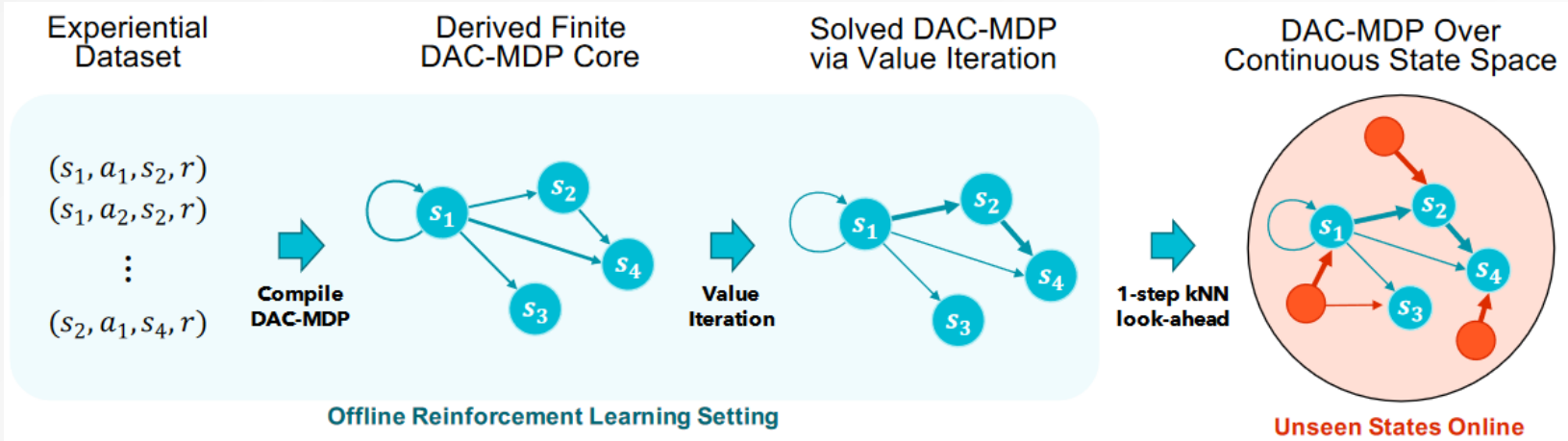
$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \beta \left(\mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \rho(\mathbf{s}, \mathbf{a})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} [Q(\mathbf{s}, \mathbf{a})] \right) + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim d_f} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right]. \quad (2)$$

Policy Improvement Using a Conservative Critic: After learning a conservative critic $\hat{Q}^\pi$, we improve the policy as:

$$\pi' \leftarrow \arg \max_{\pi} \mathbb{E}_{\mathbf{s} \sim \rho, \mathbf{a} \sim \pi(\cdot | \mathbf{s})} \left[\hat{Q}^\pi(\mathbf{s}, \mathbf{a}) \right] \quad (3)$$

DAC-MDP

Uses latent representations derived from **random projections** and those learns by **Q-iteration** algorithms



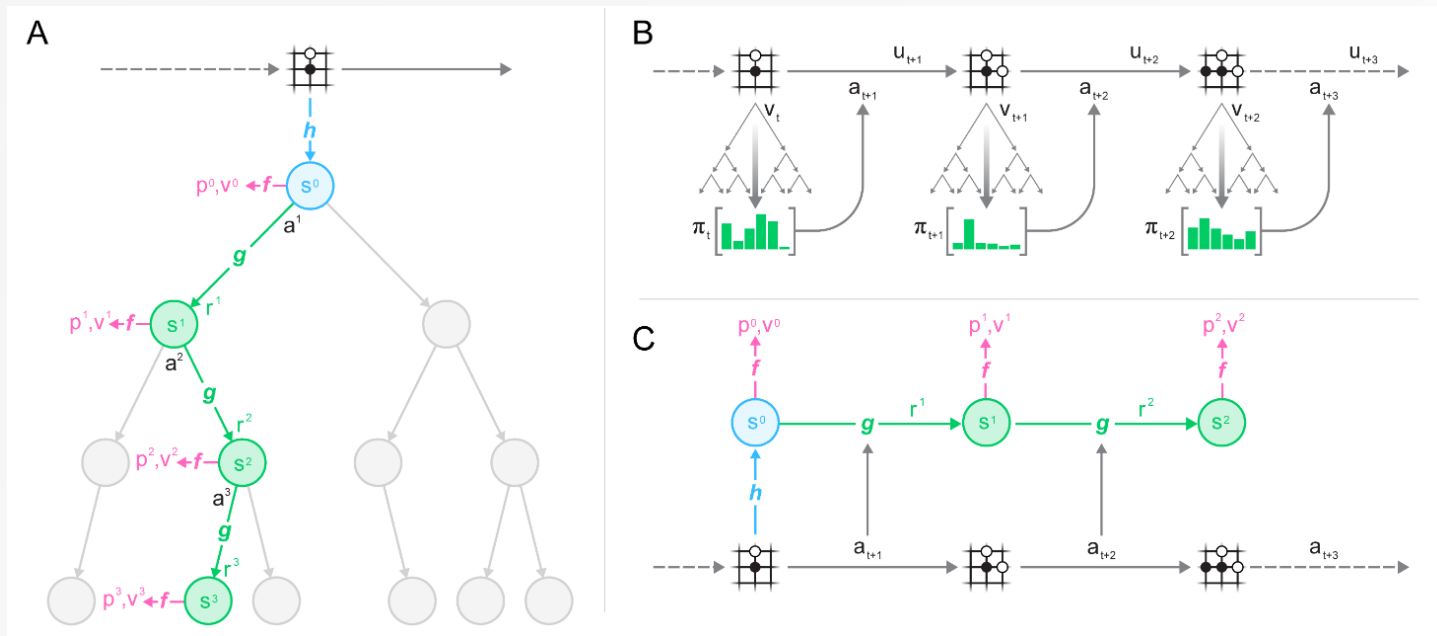
- Compile the dataset into a tabular MDP using k-nearest neighbor (kNN) queries to define the reward and transition functions.

$$\tilde{R}(s, a) = \frac{1}{k} \sum_{i \in kNN(s, a)} r_i - C \cdot d(s, a, s_i, a_i), \quad \tilde{T}(s, a, s') = \frac{1}{k} \sum_{i \in kNN(s, a)} I[s' = s'_i]$$

- Solve for the tabular MDP's optimal Q-function and define the Q-function over the entire DAC-MDP

$$\tilde{Q}(s, a) = \frac{1}{k} \sum_{i \in kNN(s, a)} r_i + \gamma \max_a \tilde{Q}(s'_i, a) - C \cdot d(s, a, s_i, a_i)$$

Combining a **tree-based search** with a **learned model**, without any knowledge of their underlying dynamics.



Appendix H Reanalyze

To improve the sample efficiency of *MuZero* we introduced a second variant of the algorithm, *MuZero Reanalyze*. *MuZero Reanalyze* revisits its past time-steps and re-executes its search using the latest model parameters, potentially resulting in a better quality policy than the original search. This fresh policy is used as the policy

MuZero Unplugged

MuZero + Reanalyse + offline RL

- MuZero: combines a tree-based search with a learned model (world model/big model, does not consider generalization/transferring)
- Reanalyse: revisits its past time-steps and **re-executes its search using the latest model parameters**, potentially resulting in a better quality policy than the original search.

A Reanalyse fraction of 0% refers to training by only interacting with the environment, no Reanalyse of stored data.

A fraction of 100% refers to the fully offline case with no environment interaction at all.

Algorithm 1 The *Reanalyse* algorithm.

```
for step  $\leftarrow$  0... $N$  do
   $t \sim \text{random}(1 : T)$ 
   $s_t = \text{representation}(o_{1:t}, \theta)$ 
  for  $i \leftarrow 0 \dots k$  do
     $\pi_t^i, \nu_t^i = \text{improve}(\text{representation}(o_{1:t+i}, \theta), \theta)$ 
     $p_t^i, v_t^i = \text{predict}(s_t^i, \theta)$ 
     $r_t^{i+1}, s_t^{i+1} = \text{dynamics}(s_t^i, \theta)$ 
  end for
   $l = \text{loss}(h_{t:t+k}, \{r, p, v, u, \pi, \nu\}_t^{0:k}, \theta)$ 
   $\Delta\theta = \text{optimise}(l, \theta)$ 
end for
```

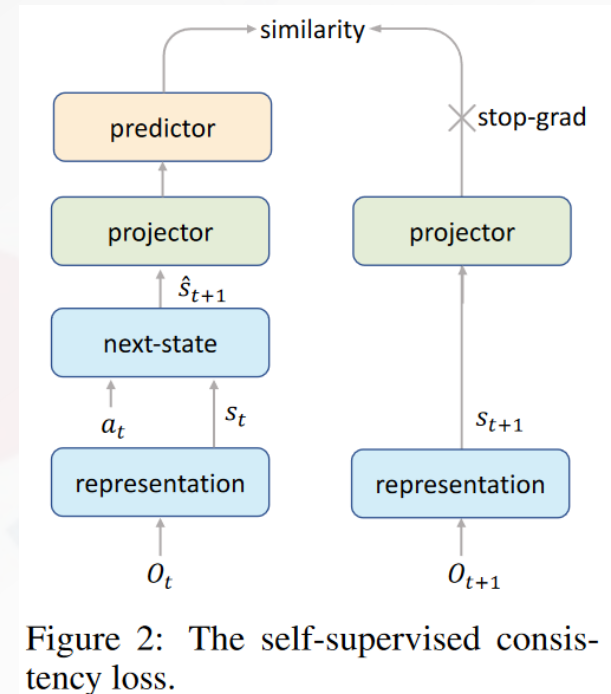
Three critical changes based on MuZero:

- (1) use self-supervised learning to learn a **temporally consistent environment model**
- (2) learn the **value prefix** in an end-to-end manner, thus helping to alleviate the compounding error in the model

$$Q(s_t, a) = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k v_{t+k}$$

- (3) use the learned model to **correct off-policy value targets**.

Redo an MCTS search with the current policy on the last state s_{t+l} and compute the empirical mean value at the root node.





THANKS

CASIA